

```
#include <iostream>

#include <cstring>

using namespace std;
```

Exercice 1

```
class pile_entier {
    int* pile;

    int taille;

    int hauteur;

public:
    // Constructeur avec paramètre par défaut
    pile_entier(int t = 20) {
        taille = t;

        hauteur = 0;

        pile = new int[taille];

        cout << "Constructeur appele. Taille = " << taille << endl;
    }

    // Destructeur
    ~pile_entier() {
        delete[] pile;

        cout << "Destructeur appele. Memoire liberee." << endl;
    }

    void afficher();

    int pleine();

    int vide();

    void empile(int valeur);

    int depile();

};

int pile_entier::pleine() {
    return hauteur == taille;
}
```

```

int pile_entier::vide() {
    return hauteur == 0;
}

void pile_entier::afficher() {
    cout << "Elements dans la pile : ";
    for (int i = 0; i < hauteur; ++i) {
        cout << pile[i] << " ";
    }
    cout << endl;
}

void pile_entier::empile(int valeur) {
    if (pleine()) {
        cout << "Erreur : La pile est pleine. Impossible d'empiler " << valeur << "." << endl;
    } else {
        pile[hauteur++] = valeur;
        cout << valeur << " empile avec succes." << endl;
    }
}

int pile_entier::depile() {
    if (vide()) {
        cout << "Erreur : La pile est vide. Rien a depiler." << endl;
        return -1;
    } else {
        return pile[--hauteur];
    }
}

//Programme principal

int main() {
    pile_entier p(5); // pile de taille 5
    p.empile(10);
}

```

```

p.empile(20);
p.empile(30);
p.afficher();
cout << "Depile : " << p.depile() << endl;
cout << "Depile : " << p.depile() << endl;
p.afficher();
cout << "Pile vide ? " << (p.vide() ? "Oui" : "Non") << endl;
cout << "Pile pleine ? " << (p.pleine() ? "Oui" : "Non") << endl;
return 0;
}

```

Exercice 2

```

class Chaine {
    char *adr;
    int longueur;
public:
    // Constructeur par défaut
    Chaine() {
        adr = new char[1];
        adr[0] = '\0';
        longueur = 0;
    }
    // Constructeur à partir d'une chaîne C
    Chaine(char *texte) {
        longueur = strlen(texte);
        adr = new char[longueur + 1];
        for(int i=0;i<longueur;i++){
            adr[i] = texte[i];
        }
        //strcpy(adr, texte);
    }
}

```

```

// Constructeur par recopie
Chaine(const Chaine& autre) {
    longueur = autre.longueur;
    adr = new char[longueur + 1];
    for(int i=0;i<longueur;i++){
        adr[i] = autre.adr[i];
    }
    //strcpy(adr, autre.adr);
}

// Destructeur
~Chaine() {
    if (adr)
        cout << "Destructeur appele pour la chaine : " << adr << endl;
    else
        cout<<"Destructeur appele pour une chaine vide\n";
    delete[] adr;
}

void affectation(const Chaine &);
int estEgal(const Chaine &);
Chaine concatener(const Chaine &);
char caractereA(int);
void afficher();
int trouver(char c);
void supprimer(int index);
void supprime(char a);
};

// Méthode d'affectation
void Chaine::affectation(const Chaine& autre) {
    if (this != &autre) {
        delete[] adr;
        longueur = autre.longueur;
    }
}

```

```

    adr = new char[longueur + 1];
    for(int i=0;i<longueur;i++){
        adr[i] = autre.adr[i];
    }
    //strcpy(adr, autre.adr);
}
adr[longueur] = '\0';
}

// Méthode de comparaison (remplace ==)
int Chaine::estEgal(const Chaine& autre) {
    int i,res=1;
    for(i=0;(i<longueur+1)&&(res!=0);i++){
        if(adr[i]!=autre.adr[i]) res=0;
    }
    return res;
    //return strcmp(adr, autre.adr) == 0;
}

// Méthode de concaténation
Chaine Chaine::concatener(const Chaine& autre) {
    Chaine res;
    res.longueur = longueur + autre.longueur;
    res.adr = new char[res.longueur+1];
    for(int i=0;i<longueur;i++){
        res.adr[i] = adr[i];
    }
    for(int i=0;i<autre.longueur;i++){
        res.adr[i+longueur] = autre.adr[i];
    }
    // Terminer la chaîne par le caractère nul
    res.adr[res.longueur]='\0';
}

```

```

return res;

/*int nouvelleLongueur = longueur + autre.longueur;
char* nouveauadr = new char[nouvelleLongueur + 1];
strcpy(nouveauadr, adr);
strcat(nouveauadr, autre.adr);

Chaine resultat(nouveauadr);
delete[] nouveauadr;
return resultat;*/
}

// Accès à un caractère
char Chaine::caractereA(int index){
    if (index >= 0 && index < longueur) {
        return adr[index];
    } else {
        cout << "Indice hors limites." << endl;
        return '\0';
    }
}

// Affichage
void Chaine::afficher() {
    for(int i=0;i<longueur;i++){
        cout<<adr[i]<<" ";
    }
    cout<<"\n";
    //cout << adr << endl;
}

int Chaine::trouver(char c) {
    for (int i = 0; i < longueur; ++i) {
        if (adr[i] == c)
            return i;
    }
}

```

```

    }
    return -1; // non trouvé
}

void Chaine::supprimer(int index) {
    if (index < 0 || index >= longueur) {
        cout<<"Out of index";
    }
    int nou = 0;
    for(int i=0; i<longueur; i++){
        if(i!=index){
            adr[nou++] = adr[i];
        }
    }
    adr[nou] = '\0';
    longueur = nou;
}

void Chaine::supprime(char a){
    int nou = 0;
    for(int i=0; i<longueur; i++){
        if(adr[i]!=a){
            adr[nou++] = adr[i];
        }
    }
    adr[nou] = '\0';
    longueur = nou;
}

int main() {
    Chaine c1("Bonjour");
    Chaine c2(" le monde");
    cout << "c1 = ";
    c1.afficher();

```

```
cout << "c2 = ";
c2.afficher();
Chaine c3 = c1.concatener(c2);
cout << "c3 (c1 + c2) = ";
c3.afficher();
cout << "Comparaison c1 et c2 : " << (c1.estEgal(c2) ? "Egales" : "Differentes") << endl;
cout << "Caractere a l'indice 1 de c1 : " << c1.caractereA(1) << endl;
Chaine c4;
c4.affectation(c1);
cout << "c4 apres affectation de c1 : ";
c4.afficher();
Chaine c5 = c4;
cout << "c5 = ";
c5.afficher();
cout << "Trouver caractere d : " << (c5.trouver('d') ? "Exist" : "Not Exist") << endl;
c5.supprimer(4);
c5.afficher();
c4.supprime('u');
cout<<"c4 apres supprimer u = ";
c4.afficher();
c3.supprimer(4);
cout<<"c3 =";
c3.afficher();
return 0; // Appelle automatiquement les destructeurs
}
```